

Chapter 8: Design a URL Shortener

Introduction

This chapter discusses the design of a URL shortening service like TinyURL. The system's main goals include **URL shortening**, **redirecting**, and **high scalability** to handle large traffic volumes.

Requirements

- Shortened URLs must be **unique** and as **short as possible**.
 - Handle **100 million URL generations per day** with a 10-year support capacity.
 - Support **efficient read operations** with a 10:1 read-to-write ratio.
 - Store 365 billion records, requiring approximately **365 TB** of storage over 10 years.
-

Step 1: High-Level Design

API Endpoints

1. URL Shortening:

- Endpoint: `POST api/v1/data/shorten`
- Parameters: `{longUrl: longURLString}`
- Returns: `shortURL`

2. URL Redirecting:

- Endpoint: `GET api/v1/shortUrl`
- Returns: `longURL` for redirection.

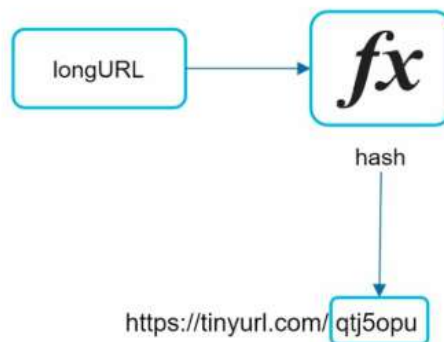
```
Request URL: https://tinyurl.com/qtj5opu
Request Method: GET
Status Code: 301
Remote Address: [2606:4700:10::6814:391e]:443
Referrer Policy: no-referrer-when-downgrade

Response Headers
alt-svc: h3-27=":443"; ma=86400, h3-25=":443"; ma=86400, h3-24=":443"; ma=86400, h3-23=":443"; ma=86400
cache-control: max-age=0, no-cache, private
cf-cache-status: DYNAMIC
cf-ray: 581fbd8ac986ed33-SJC
content-type: text/html; charset=UTF-8
date: Fri, 10 Apr 2020 22:00:23 GMT
expect-ct: max-age=604800, report-uri="https://report-uri.cloudflare.com/cdn-cgi/beacon/expect-ct"
location: https://www.amazon.com/dp/B017V4NTFA?pf_rd_p=63eef76-979c-4d&ref=adblp13nvxx_0_2_im
```

URL Redirection

- **301 Redirect:** A 301 redirect shows that the requested URL is “permanently” moved to the long URL. The browser caches the response, and subsequent requests for the same URL will not be sent to the URL shortening service.
- **302 Redirect:** Temporary; useful for analytics like tracking clicks.

URL Shortening



- Use a **hash function** to generate a short URL, mapping long URLs to unique shortened versions.
- The hash function must satisfy the following requirements:
 - Each longURL must be hashed to one hashValue.
 - Each hashValue can be mapped back to the longURL.

Step 2: Deep Dive into Design

Data Model

Store `<shortURL, longURL>` mappings in a relational database to optimize memory usage. The table schema includes:

- `id` (primary key),
- `shortURL` ,
- `longURL` .

url Table	
PK	<u>id (auto increment)</u>
	shortURL longURL

Hash Function

1. Base 62 Conversion:

- Encodes numbers using characters `[0-9, a-z, A-Z]` , providing **62 possible characters**.
- Base conversion is another approach commonly used for URL shorteners.
- A unique id can be assigned to the short url and ID can be base 62 converted to get the short URL.
- A 7-character hash supports up to **3.5 trillion unique URLs**, enough for 365 billion URLs.

Example:

Convert ID `2009215674938` to Base 62:

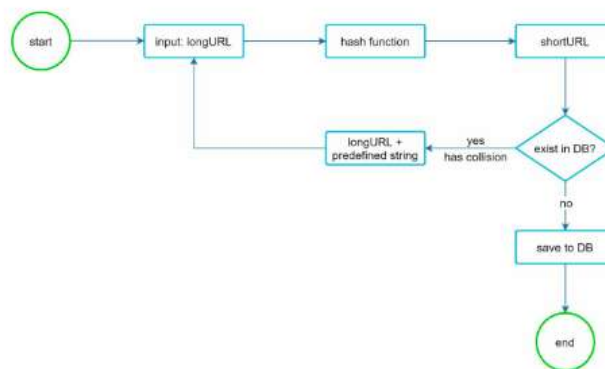
- `2009215674938` → `zn9edcu` .

2. Hash + Collision Resolution:

- Use hash functions like CRC32, MD5, or SHA-1.

Hash function	Hash value (Hexadecimal)
CRC32	5cb54054
MD5	5a62509a84df9ee03fe1230b9df8b84e
SHA-1	0eeae7916c06853901d9ccbefbcaf4de57ed85b

- One approach is to collect the first 7 characters of a hash value; however, this method can lead to hash collisions.
- To resolve collisions, recursively append a new predefined string until no more collision but this can be expensive.
- Resolve collisions with **Bloom Filters** for efficient lookup.

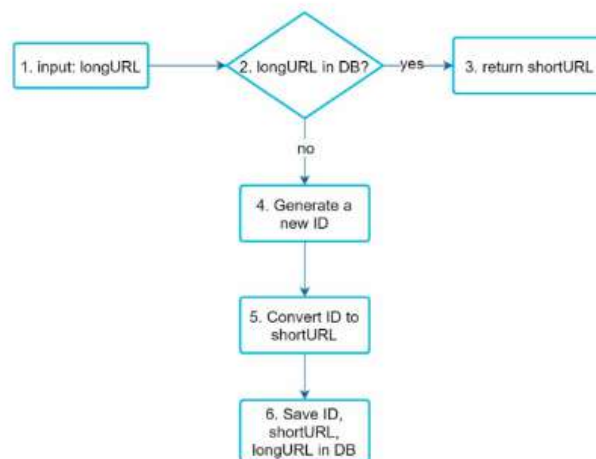


Comparison

- **Hash + Collision Resolution:**
 - Fixed short URL length
 - Does not need a unique ID generator
 - Collision is possible and needs resolution
 - Not possible to find the next available short URL because it does not depend on ID
- **Base 62 Conversion**
 - The length is not fixed and goes up with ID
 - It needs a unique ID generator
 - Collision is not possible

- Easy to find the next short URL if ID increments by 1 (Can be a security concern)

URL Shortening Flow



1. Check if `longURL` exists in the database.
2. If found, return the existing `shortURL`.
3. Otherwise:
 - Generate a unique ID using a **distributed ID generator**.
 - Convert the ID to `shortURL` using Base 62.
 - Store the `<id, shortURL, longURL>` mapping in the database.

URL Redirecting Flow



1. User clicks a `shortURL`.
2. Query `<shortURL, longURL>` mapping:
 - Check the `cache` first for faster access.
 - If not in the cache, query the database.
3. Redirect the user to `longURL`.

Additional Considerations

Rate Limiter

- Prevent abuse by setting limits on requests per IP.

Scalability

1. **Web Tier:** Stateless, scalable by adding/removing web servers.
2. **Database Tier:** Use replication and sharding.

Analytics

- Collect data like click rates, source, and timestamps for business insights.

High Availability and Reliability

- Ensure consistent and reliable services using database replication and fault-tolerant design.